

This is called **stacking**, which stands for “stacked generalization” [Wol92]. Note that the combination weights used by stacking need to be trained on a separate dataset, otherwise they would put all their mass on the best performing base model.

### 18.2.2 Ensembling is not Bayes model averaging

It is worth noting that an ensemble of models is not the same as using Bayes model averaging over models (Section 4.6), as pointed out in [Min00]. An ensemble considers a larger hypothesis class of the form

$$p(y|\mathbf{x}, \mathbf{w}, \boldsymbol{\theta}) = \sum_{m \in \mathcal{M}} w_m p(y|\mathbf{x}, \boldsymbol{\theta}_m) \quad (18.13)$$

whereas BMA uses

$$p(y|\mathbf{x}, \mathcal{D}) = \sum_{m \in \mathcal{M}} p(m|\mathcal{D}) p(y|\mathbf{x}, m, \mathcal{D}) \quad (18.14)$$

The key difference is that in the case of BMA, the weights  $p(m|\mathcal{D})$  sum to one, and in the limit of infinite data, only a single model will be chosen (namely the MAP model). By contrast, the ensemble weights  $w_m$  are arbitrary, and don’t collapse in this way to a single model.

## 18.3 Bagging

In this section, we discuss **bagging** [Bre96], which stands for “bootstrap aggregating”. This is a simple form of ensemble learning in which we fit  $M$  different base models to different randomly sampled versions of the data; this encourages the different models to make diverse predictions. The datasets are sampled with replacement (a technique known as bootstrap sampling, Section 4.7.3), so a given example may appear multiple times, until we have a total of  $N$  examples per model (where  $N$  is the number of original data points).

The disadvantage of bootstrap is that each base model only sees, on average, 63% of the unique input examples. To see why, note that the chance that a single item will not be selected from a set of size  $N$  in any of  $N$  draws is  $(1 - 1/N)^N$ . In the limit of large  $N$ , this becomes  $e^{-1} \approx 0.37$ , which means only  $1 - 0.37 = 0.63$  of the data points will be selected.

The 37% of the training instances that are not used by a given base model are called **out-of-bag instances** (oob). We can use the predicted performance of the base model on these oob instances as an estimate of test set performance. This provides a useful alternative to cross validation.

The main advantage of bootstrap is that it prevents the ensemble from relying too much on any individual training example, which enhances robustness and generalization [Gra04]. For example, comparing Figure 18.3b and Figure 18.3c, we see that omitting a single example from the training set can have a large impact on the decision tree that we learn (even though the tree growing algorithm is otherwise deterministic). By averaging the predictions from both of these models, we get the more reasonable prediction model in Figure 18.3d. This advantage generally increases with the size of the ensemble, as shown in Figure 18.4. (Of course, larger ensembles take more memory and more time.)

Bagging does not always improve performance. In particular, it relies on the base models being unstable estimators, so that omitting some of the data significantly changes the resulting model fit.

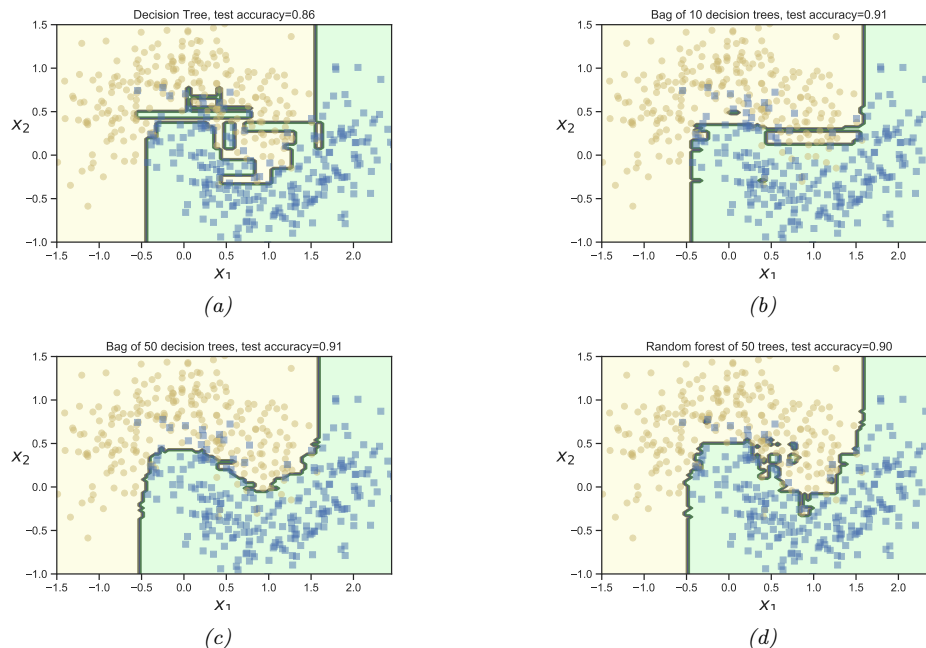


Figure 18.4: (a) A single decision tree. (b-c) Bagging ensemble of 10 and 50 trees. (d) Random forest of 50 trees. Adapted from Figure 7.5 of [Gér19]. Generated by [bagging\\_trees.ipynb](#) and [rf\\_demo\\_2d.ipynb](#).

This is the case for decision trees, but not for other models, such as nearest neighbor classifiers. For neural networks, the story is more mixed. They can be unstable wrt their training set. On the other hand, deep networks will underperform if they only see 63% of the data, so bagged DNNs do not usually work well [NTL20].

## 18.4 Random forests

Bagging relies on the assumption that re-running the same learning algorithm on different subsets of the data will result in sufficiently diverse base models. The technique known as **random forests** [Bre01] tries to decorrelate the base learners even further by learning trees based on a randomly chosen subset of input variables (at each node of the tree), as well as a randomly chosen subset of data cases. It does this by modifying Equation (18.5) so the the feature split dimension  $j$  is optimized over a random subset of the features,  $S_i \subset \{1, \dots, D\}$ .

For example, consider the email spam dataset [HTF09, p301]. This dataset contains 4601 email messages, each of which is classified as spam (1) or non-spam (0). The data was open sourced by George Forman from Hewlett-Packard (HP) Labs.

There are 57 quantitative (real-valued) features, as follows:

- 48 features corresponding to the percentage of words in the email that match a given word, such as “remove” or “labs”.

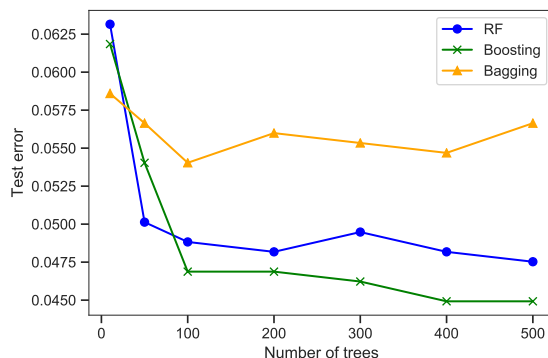


Figure 18.5: Predictive accuracy vs size of tree ensemble for bagging, random forests and gradient boosting with log loss. Adapted from Figure 15.1 of [HTF09]. Generated by `spam_tree_ensemble_compare.ipynb`.

- 6 features corresponding to the percentage of characters in the email that match a given character, namely ; . [ ! \$ #
- 3 features corresponding to the average length, max length, and sum of lengths of uninterrupted sequences of capital letters. (These features are called CAPAVE, CAPMAX and CAPTOT.)

Figure 18.5 shows that random forests work much better than bagged decision trees, because many input features are irrelevant. (We also see that a method called “boosting”, discussed in Section 18.5, works even better; however, this requires sequentially fitting trees, whereas random forests can be fit in parallel.)

## 18.5 Boosting

Ensembles of trees, whether fit by bagging or the random forest algorithm, corresponding to a model of the form

$$f(\mathbf{x}; \boldsymbol{\theta}) = \sum_{m=1}^M \beta_m F_m(\mathbf{x}; \boldsymbol{\theta}_m) \quad (18.15)$$

where  $F_m$  is the  $m$ 'th tree, and  $\beta_m$  is the corresponding weight, often set to  $\beta_m = 1/M$ . We can generalize this by allowing the  $F_m$  functions to be general function approximators, such as neural networks, not just trees. The result is called an **additive model** [HTF09]. We can think of this as a linear model with **adaptive basis functions**. The goal, as usual, is to minimize the empirical loss (with an optional regularizer):

$$\mathcal{L}(f) = \sum_{i=1}^N \ell(y_i, f(\mathbf{x}_i)) \quad (18.16)$$

**Boosting** [Sch90; FS96] is an algorithm for sequentially fitting additive models where each  $F_m$  is a binary classifier that returns  $F_m \in \{-1, +1\}$ . In particular, we first fit  $F_1$  on the original data,

and then we weight the data samples by the errors made by  $F_1$ , so misclassified examples get more weight. Next we fit  $F_2$  to this weighted data set. We keep repeating this process until we have fit the desired number  $M$  of components. ( $M$  is a hyper-parameter that controls the complexity of the overall model, and can be chosen by monitoring performance on a validation set, and using early stopping.)

It can be shown that, as long as each  $F_m$  has an accuracy that is better than chance (even on the weighted dataset), then the final ensemble of classifiers will have higher accuracy than any given component. That is, if  $F_m$  is a **weak learner** (so its accuracy is only slightly better than 50%), then we can boost its performance using the above procedure so that the final  $f$  becomes a **strong learner**. (See e.g., [SF12] for more details on the learning theory approach to boosting.)

Note that boosting reduces the bias of the strong learner, by fitting trees that depend on each other, whereas bagging and RF reduce the variance by fitting independent trees. In many cases, boosting can work better. See Figure 18.5 for an example.

The original boosting algorithm focused on binary classification with a particular loss function that we will explain in Section 18.5.3, and was derived from the PAC learning theory framework (see Section 5.4.4). In the rest of this section, we focus on a more statistical version of boosting, due to [FHT00; Fri01], which works with arbitrary loss functions, making the method suitable for regression, multi-class classification, ranking, etc. Our presentation is based on [HTF09, ch10] and [BH07], which should be consulted for further details.

### 18.5.1 Forward stagewise additive modeling

In this section, we discuss **forward stagewise additive modeling**, in which we sequentially optimize the objective in Equation (18.16) for general (differentiable) loss functions, where  $f$  is an additive model as in Equation 18.15. That is, at iteration  $m$ , we compute

$$(\beta_m, \theta_m) = \underset{\beta, \theta}{\operatorname{argmin}} \sum_{i=1}^N \ell(y_i, f_{m-1}(\mathbf{x}_i) + \beta F(\mathbf{x}_i; \theta)) \quad (18.17)$$

We then set

$$f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \beta_m F(\mathbf{x}; \theta_m) = f_{m-1}(\mathbf{x}) + \beta_m F_m(\mathbf{x}) \quad (18.18)$$

(Note that we do not adjust the parameters of previously added models.) The details on how to perform this optimization step depend on the loss function that we choose, and (in some cases) on the form of the weak learner  $F$ , as we discuss below.

### 18.5.2 Quadratic loss and least squares boosting

Suppose we use squared error loss,  $\ell(y, \hat{y}) = (y - \hat{y})^2$ . In this case, the  $i$ 'th term in the objective at step  $m$  becomes

$$\ell(y_i, f_{m-1}(\mathbf{x}_i) + \beta F(\mathbf{x}_i; \theta)) = (y_i - f_{m-1}(\mathbf{x}_i) - \beta F(\mathbf{x}_i; \theta))^2 = (r_{im} - \beta F(\mathbf{x}_i; \theta))^2 \quad (18.19)$$

where  $r_{im} = y_i - f_{m-1}(\mathbf{x}_i)$  is the residual of the current model on the  $i$ 'th observation. We can minimize the above objective by simply setting  $\beta = 1$ , and fitting  $F$  to the residual errors. This is called **least squares boosting** [BY03].

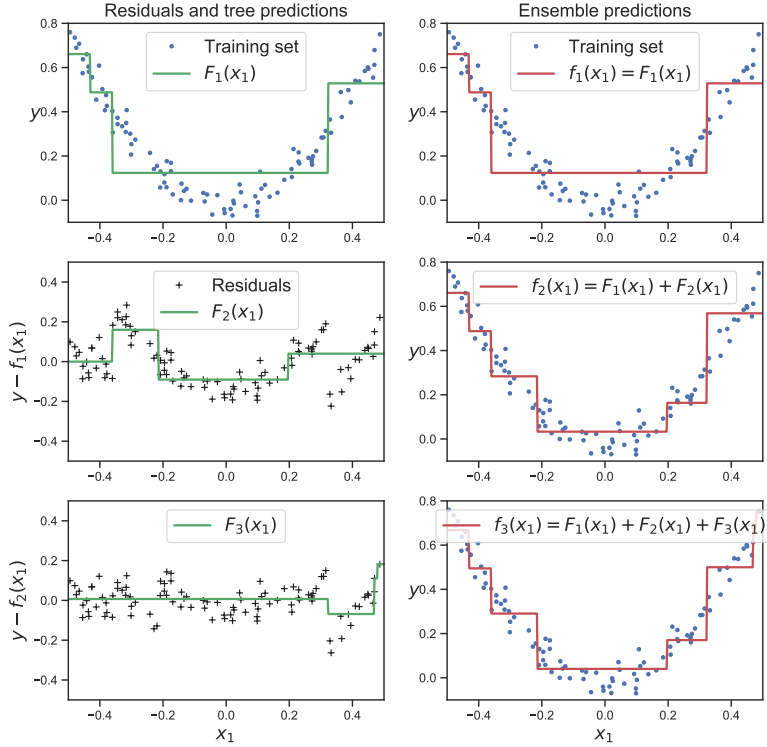


Figure 18.6: Illustration of boosting using a regression tree of depth 2 applied to a 1d dataset. Adapted from Figure 7.9 of [Gér19]. Generated by `boosted_regr_trees.ipynb`.

We give an example of this process in Figure 18.6, where we use a regression tree of depth 2 as the weak learner. On the left, we show the result of fitting the weak learner to the residuals, and on the right, we show the current strong learner. We see how each new weak learner that is added to the ensemble corrects the errors made by earlier versions of the model.

### 18.5.3 Exponential loss and AdaBoost

Suppose we are interested in binary classification, i.e., predicting  $\tilde{y}_i \in \{-1, +1\}$ . Let us assume the weak learner computes

$$p(y = 1|\mathbf{x}) = \frac{e^{F(\mathbf{x})}}{e^{-F(\mathbf{x})} + e^{F(\mathbf{x})}} = \frac{1}{1 + e^{-2F(\mathbf{x})}} \quad (18.20)$$

so  $F(\mathbf{x})$  returns half the log odds. We know from Equation (10.13) that the negative log likelihood is given by

$$\ell(\tilde{y}, F(\mathbf{x})) = \log(1 + e^{-2\tilde{y}F(\mathbf{x})}) \quad (18.21)$$

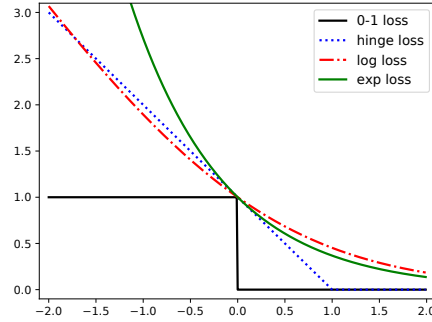


Figure 18.7: Illustration of various loss functions for binary classification. The horizontal axis is the margin  $m(\mathbf{x}) = \tilde{y}F(\mathbf{x})$ , the vertical axis is the loss. The log loss uses log base 2. Generated by [hinge\\_loss\\_plot.ipynb](#).

We can minimize this by ensuring that the **margin**  $m(\mathbf{x}) = \tilde{y}F(\mathbf{x})$  is as large as possible. We see from Figure 18.7 that the log loss is a smooth upper bound on the 0-1 loss. We also see that it penalizes negative margins more heavily than positive ones, as desired (since positive margins are already correctly classified).

However, we can also use other loss functions. In this section, we consider the **exponential loss**

$$\ell(\tilde{y}, F(\mathbf{x})) = \exp(-\tilde{y}F(\mathbf{x})) \quad (18.22)$$

We see from Figure 18.7 that this is also a smooth upper bound on the 0-1 loss. In the population setting (with infinite sample size), the optimal solution to the exponential loss is the same as for log loss. To see this, we can just set the derivative of the expected loss (for each  $\mathbf{x}$ ) to zero:

$$\frac{\partial}{\partial F(\mathbf{x})} \mathbb{E} \left[ e^{-\tilde{y}f(\mathbf{x})} | \mathbf{x} \right] = \frac{\partial}{\partial F(\mathbf{x})} [p(\tilde{y} = 1 | \mathbf{x})e^{-F(\mathbf{x})} + p(\tilde{y} = -1 | \mathbf{x})e^{F(\mathbf{x})}] \quad (18.23)$$

$$= -p(\tilde{y} = 1 | \mathbf{x})e^{-F(\mathbf{x})} + p(\tilde{y} = -1 | \mathbf{x})e^{F(\mathbf{x})} \quad (18.24)$$

$$= 0 \Rightarrow \frac{p(\tilde{y} = 1 | \mathbf{x})}{p(\tilde{y} = -1 | \mathbf{x})} = e^{2F(\mathbf{x})} \quad (18.25)$$

However, it turns out that the exponential loss is easier to optimize in the boosting setting, as we show below. (We consider the log loss case in Section 18.5.4.)

We now discuss how to solve for the  $m$ 'th weak learner,  $F_m$ , when we use exponential loss. We will assume that the base classifier  $F_m$  returns a binary class label; the resulting algorithm is called **discrete AdaBoost** [FHT00]. If  $F_m$  returns a probability instead, a modified algorithm, known as **real AdaBoost**, can be used [FHT00].

At step  $m$  we have to minimize

$$L_m(F) = \sum_{i=1}^N \exp[-\tilde{y}_i(f_{m-1}(\mathbf{x}_i) + \beta F(\mathbf{x}_i))] = \sum_{i=1}^N \omega_{i,m} \exp(-\beta \tilde{y}_i F(\mathbf{x}_i)) \quad (18.26)$$

where  $\omega_{i,m} \triangleq \exp(-\tilde{y}_i f_{m-1}(\mathbf{x}_i))$  is a weight applied to datacase  $i$ , and  $\tilde{y}_i \in \{-1, +1\}$ . We can rewrite this objective as follows:

$$L_m = e^{-\beta} \sum_{\tilde{y}_i = F(\mathbf{x}_i)} \omega_{i,m} + e^{\beta} \sum_{\tilde{y}_i \neq F(\mathbf{x}_i)} \omega_{i,m} \quad (18.27)$$

$$= (e^{\beta} - e^{-\beta}) \sum_{i=1}^N \omega_{i,m} \mathbb{I}(\tilde{y}_i \neq F(\mathbf{x}_i)) + e^{-\beta} \sum_{i=1}^N \omega_{i,m} \quad (18.28)$$

Consequently the optimal function to add is

$$F_m = \operatorname{argmin}_F \sum_{i=1}^N \omega_{i,m} \mathbb{I}(\tilde{y}_i \neq F(\mathbf{x}_i)) \quad (18.29)$$

This can be found by applying the weak learner to a weighted version of the dataset, with weights  $\omega_{i,m}$ .

All that remains is to solve for the size of the update,  $\beta$ . Substituting  $F_m$  into  $L_m$  and solving for  $\beta$  we find

$$\beta_m = \frac{1}{2} \log \frac{1 - \text{err}_m}{\text{err}_m} \quad (18.30)$$

where

$$\text{err}_m = \frac{\sum_{i=1}^N \omega_{i,m} \mathbb{I}(\tilde{y}_i \neq F_m(\mathbf{x}_i))}{\sum_{i=1}^N \omega_{i,m}} \quad (18.31)$$

Therefore overall update becomes

$$f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \beta_m F_m(\mathbf{x}) \quad (18.32)$$

After updating the strong learner, we need to recompute the weights for the next iteration, as follows:

$$\omega_{i,m+1} = e^{-\tilde{y}_i f_m(\mathbf{x}_i)} = e^{-\tilde{y}_i f_{m-1}(\mathbf{x}_i) - \tilde{y}_i \beta_m F_m(\mathbf{x}_i)} = \omega_{i,m} e^{-\tilde{y}_i \beta_m F_m(\mathbf{x}_i)} \quad (18.33)$$

If  $\tilde{y}_i = F_m(\mathbf{x}_i)$ , then  $\tilde{y}_i F_m(\mathbf{x}_i) = 1$ , and if  $\tilde{y}_i \neq F_m(\mathbf{x}_i)$ , then  $\tilde{y}_i F_m(\mathbf{x}_i) = -1$ . Hence  $-\tilde{y}_i F_m(\mathbf{x}_i) = 2\mathbb{I}(\tilde{y}_i \neq F_m(\mathbf{x}_i)) - 1$ , so the update becomes

$$\omega_{i,m+1} = \omega_{i,m} e^{\beta_m (2\mathbb{I}(\tilde{y}_i \neq F_m(\mathbf{x}_i)) - 1)} = \omega_{i,m} e^{2\beta_m \mathbb{I}(\tilde{y}_i \neq F_m(\mathbf{x}_i))} e^{-\beta_m} \quad (18.34)$$

Since the  $e^{-\beta_m}$  is constant across all examples, it can be dropped. If we then define  $\alpha_m = 2\beta_m$ , the update becomes

$$\omega_{i,m+1} = \begin{cases} \omega_{i,m} e^{\alpha_m} & \text{if } \tilde{y}_i \neq F_m(\mathbf{x}_i) \\ \omega_{i,m} & \text{otherwise} \end{cases} \quad (18.35)$$

Thus we see that we exponentially increase weights of misclassified examples. The resulting algorithm shown in Algorithm 18.1, and is known as **Adaboost.M1** [FS96].

A multiclass generalization of exponential loss, and an adaboost-like algorithm to minimize it, known as **SAMME** (stagewise additive modeling using a multiclass exponential loss function), is described in [Has+09]. This is implemented in scikit learn (the AdaBoostClassifier class).

---

**Algorithm 18.1:** Adaboost.M1, for binary classification with exponential loss

---

```

1  $\omega_i = 1/N$ 
2 for  $m = 1 : M$  do
3   Fit a classifier  $F_m(\mathbf{x})$  to the training set using weights  $\mathbf{w}$ 
4   Compute  $\text{err}_m = \frac{\sum_{i=1}^N \omega_{i,m} \mathbb{I}(\tilde{y}_i \neq F_m(\mathbf{x}_i))}{\sum_{i=1}^N \omega_{i,m}}$ 
5   Compute  $\alpha_m = \log[(1 - \text{err}_m)/\text{err}_m]$ 
6   Set  $\omega_i \leftarrow \omega_i \exp[\alpha_m \mathbb{I}(\tilde{y}_i \neq F_m(\mathbf{x}_i))]$ 
7 Return  $f(\mathbf{x}) = \text{sgn} \left[ \sum_{m=1}^M \alpha_m F_m(\mathbf{x}) \right]$ 

```

---

### 18.5.4 LogitBoost

The trouble with exponential loss is that it puts a lot of weight on misclassified examples, as is apparent from the exponential blowup on the left hand side of Figure 18.7. This makes the method very sensitive to outliers (misabeled examples). In addition,  $e^{-\tilde{y}f}$  is not the logarithm of any pmf for binary variables  $\tilde{y} \in \{-1, +1\}$ ; consequently we cannot recover probability estimates from  $f(\mathbf{x})$ .

A natural alternative is to use log loss, as we discussed in Section 18.5.3. This only punishes mistakes linearly, as is clear from Figure 18.7. Furthermore, it means that we will be able to extract probabilities from the final learned function, using

$$p(y = 1|\mathbf{x}) = \frac{e^{f(\mathbf{x})}}{e^{-f(\mathbf{x})} + e^{f(\mathbf{x})}} = \frac{1}{1 + e^{-2f(\mathbf{x})}} \quad (18.36)$$

The goal is to minimize the expected log-loss, given by

$$L_m(F) = \sum_{i=1}^N \log [1 + \exp (-2\tilde{y}_i(f_{m-1}(\mathbf{x}) + F(\mathbf{x}_i)))] \quad (18.37)$$

By performing a Newton update on this objective (similar to IRLS), one can derive the algorithm shown in Algorithm 18.2. This is known as **logitBoost** [FHT00]. The key subroutine is the ability of the weak learner  $F$  to solve a weighted least squares problem. This method can be generalized to the multi-class setting, as explained in [FHT00].

### 18.5.5 Gradient boosting

Rather than deriving new versions of boosting for every different loss function, it is possible to derive a generic version, known as **gradient boosting** [Fri01; Mas+00]. To explain this, imagine solving  $\hat{\mathbf{f}} = \text{argmin}_{\mathbf{f}} \mathcal{L}(\mathbf{f})$  by performing gradient descent in the space of functions. Since functions are infinite dimensional objects, we will represent them by their values on the training set,  $\mathbf{f} = (f(\mathbf{x}_1), \dots, f(\mathbf{x}_N))$ . At step  $m$ , let  $\mathbf{g}_m$  be the gradient of  $\mathcal{L}(\mathbf{f})$  evaluated at  $\mathbf{f} = \mathbf{f}_{m-1}$ :

$$g_{im} = \left[ \frac{\partial \ell(y_i, f(\mathbf{x}_i))}{\partial f(\mathbf{x}_i)} \right]_{f=f_{m-1}} \quad (18.38)$$

**Algorithm 18.2:** LogitBoost, for binary classification with log-loss

---

```

1  $\omega_i = 1/N, \pi_i = 1/2$ 
2 for  $m = 1 : M$  do
3   Compute the working response  $z_i = \frac{y_i^* - \pi_i}{\pi_i(1 - \pi_i)}$ 
4   Compute the weights  $\omega_i = \pi_i(1 - \pi_i)$ 
5    $F_m = \operatorname{argmin}_F \sum_{i=1}^N \omega_i (z_i - F(\mathbf{x}_i))^2$ 
6   Update  $f(\mathbf{x}) \leftarrow f(\mathbf{x}) + \frac{1}{2} F_m(\mathbf{x})$ 
7   Compute  $\pi_i = 1/(1 + \exp(-2f(\mathbf{x}_i)))$ ;
8 Return  $f(\mathbf{x}) = \operatorname{sgn} \left[ \sum_{m=1}^M F_m(\mathbf{x}) \right]$ 

```

---

| Name               | Loss                                   | $-\partial \ell(y_i, f(\mathbf{x}_i)) / \partial f(\mathbf{x}_i)$ |
|--------------------|----------------------------------------|-------------------------------------------------------------------|
| Squared error      | $\frac{1}{2}(y_i - f(\mathbf{x}_i))^2$ | $y_i - f(\mathbf{x}_i)$                                           |
| Absolute error     | $ y_i - f(\mathbf{x}_i) $              | $\operatorname{sgn}(y_i - f(\mathbf{x}_i))$                       |
| Exponential loss   | $\exp(-\tilde{y}_i f(\mathbf{x}_i))$   | $-\tilde{y}_i \exp(-\tilde{y}_i f(\mathbf{x}_i))$                 |
| Binary Logloss     | $\log(1 + e^{-\tilde{y}_i f_i})$       | $y_i - \pi_i$                                                     |
| Multiclass logloss | $-\sum_c y_{ic} \log \pi_{ic}$         | $y_{ic} - \pi_{ic}$                                               |

Table 18.1: Some commonly used loss functions and their gradients. For binary classification problems, we assume  $\tilde{y}_i \in \{-1, +1\}$ , and  $\pi_i = \sigma(2f(\mathbf{x}_i))$ . For regression problems, we assume  $y_i \in \mathbb{R}$ . Adapted from [HTF09, p360] and [BH07, p483].

Gradients of some common loss functions are given in Table 18.1. We then make the update

$$\mathbf{f}_m = \mathbf{f}_{m-1} - \beta_m \mathbf{g}_m \quad (18.39)$$

where  $\beta_m$  is the step length, chosen by

$$\beta_m = \operatorname{argmin}_{\beta} \mathcal{L}(\mathbf{f}_{m-1} - \beta \mathbf{g}_m) \quad (18.40)$$

In its current form, this is not much use, since it only optimizes  $f$  at a fixed set of  $N$  points, so we do not learn a function that can generalize. However, we can modify the algorithm by fitting a weak learner to approximate the negative gradient signal. That is, we use this update

$$F_m = \operatorname{argmin}_F \sum_{i=1}^N (-g_{im} - F(\mathbf{x}_i))^2 \quad (18.41)$$

The overall algorithm is summarized in Algorithm 18.3. We have omitted the line search step for  $\beta_m$ , which is not strictly necessary, as argued in [BH07]. However, we have introduced a learning rate or **shrinkage factor**  $0 < \nu \leq 1$ , to control the size of the updates, for regularization purposes.

If we apply this algorithm using squared loss, we recover L2Boosting, since  $-g_{im} = y_i - f_{m-1}(\mathbf{x}_i)$  is just the residual error. We can also apply this algorithm to other loss functions, such as absolute loss or Huber loss (Section 5.1.5.3), which is useful for robust regression problems.

**Algorithm 18.3:** Gradient boosting

---

```

1 Initialize  $f_0(\mathbf{x}) = \operatorname{argmin}_F \sum_{i=1}^N L(y_i, F(\mathbf{x}_i))$ 
2 for  $m = 1 : M$  do
3   Compute the gradient residual using  $r_{im} = - \left[ \frac{\partial L(y_i, f(\mathbf{x}_i))}{\partial f(\mathbf{x}_i)} \right]_{f(\mathbf{x}_i)=f_{m-1}(\mathbf{x}_i)}$ 
4   Use the weak learner to compute  $F_m = \operatorname{argmin}_F \sum_{i=1}^N (r_{im} - F(\mathbf{x}_i))^2$ 
5   Update  $f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \nu F_m(\mathbf{x})$ 
6 Return  $f(\mathbf{x}) = f_M(\mathbf{x})$ 

```

---

For classification, we can use log-loss. In this case, we get an algorithm known as **BinomialBoost** [BH07]. The advantage of this over LogitBoost is that it does not need to be able to do weighted fitting: it just applies any black-box regression model to the gradient vector. To apply this to multi-class classification, we can fit  $C$  separate regression trees, using the pseudo residual of the form

$$-g_{icm} = \frac{\partial \ell(y_i, f_{1m}(\mathbf{x}_i), \dots, f_{Cm}(\mathbf{x}_i))}{\partial f_{cm}(\mathbf{x}_i)} = \mathbb{I}(y_i = c) - \pi_{ic} \quad (18.42)$$

Although the trees are fit separately, their predictions are combined via a softmax transform

$$p(y = c | \mathbf{x}) = \frac{e^{f_c(\mathbf{x})}}{\sum_{c'=1}^C e^{f_{c'}(\mathbf{x})}} \quad (18.43)$$

When we have large datasets, we can use a stochastic variant in which we subsample (without replacement) a random fraction of the data to pass to the regression tree at each iteration. This is called **stochastic gradient boosting** [Fri99]. Not only is it faster, but it can also generalize better, because subsampling the data is a form of regularization.

### 18.5.5.1 Gradient tree boosting

In practice, gradient boosting nearly always assumes that the weak learner is a regression tree, which is a model of the form

$$F_m(\mathbf{x}) = \sum_{j=1}^{J_m} w_{jm} \mathbb{I}(\mathbf{x} \in R_{jm}) \quad (18.44)$$

where  $w_{jm}$  is the predicted output for region  $R_{jm}$ . (In general,  $w_{jm}$  could be a vector.) This combination is called **gradient boosted regression trees**, or **gradient tree boosting**. (A related version is known as **MART**, which stands for “multivariate additive regression trees” [FM03].)

To use this in gradient boosting, we first find good regions  $R_{jm}$  for tree  $m$  using standard regression tree learning (see Section 18.1) on the residuals; we then (re)solve for the weights of each leaf by solving

$$\hat{w}_{jm} = \operatorname{argmin}_w \sum_{\mathbf{x}_i \in R_{jm}} \ell(y_i, f_{m-1}(\mathbf{x}_i) + w) \quad (18.45)$$

For squared error (as used by gradient boosting), the optimal weight  $\hat{w}_{jm}$  is just the mean of the residuals in that leaf.

### 18.5.5.2 XGBoost

**XGBoost** (<https://github.com/dmlc/xgboost>), which stands for “extreme gradient boosting”, is a very efficient and widely used implementation of gradient boosted trees, that adds a few more improvements beyond the description in Section 18.5.5.1. The details can be found in [CG16], but in brief, the extensions are as follows: it adds a regularizer on the tree complexity, it uses a second order approximation of the loss (from [FHT00]) instead of just a linear approximation, it samples features at internal nodes (as in random forests), and it uses various computer science methods (such as handling out-of-core computation for large datasets) to ensure scalability.<sup>2</sup>

In more detail, XGBoost optimizes the following regularized objective

$$\mathcal{L}(f) = \sum_{i=1}^N \ell(y_i, f(\mathbf{x}_i)) + \Omega(f) \quad (18.46)$$

where

$$\Omega(f) = \gamma J + \frac{1}{2} \lambda \sum_{j=1}^J w_j^2 \quad (18.47)$$

is the regularizer, where  $J$  is the number of leaves, and  $\gamma \geq 0$  and  $\lambda \geq 0$  are regularization coefficients. At the  $m$ 'th step, the loss is given by

$$\mathcal{L}_m(F_m) = \sum_{i=1}^N \ell(y_i, f_{m-1}(\mathbf{x}_i) + F_m(\mathbf{x}_i)) + \Omega(F_m) + \text{const} \quad (18.48)$$

We can compute a second order Taylor expansion of this as follows:

$$\mathcal{L}_m(F_m) \approx \sum_{i=1}^N \left[ \ell(y_i, f_{m-1}(\mathbf{x}_i)) + g_{im} F_m(\mathbf{x}_i) + \frac{1}{2} h_{im} F_m^2(\mathbf{x}_i) \right] + \Omega(F_m) + \text{const} \quad (18.49)$$

where  $h_{im}$  is the Hessian

$$h_{im} = \left[ \frac{\partial^2 \ell(y_i, f(\mathbf{x}_i))}{\partial f(\mathbf{x}_i)^2} \right]_{f=f_{m-1}} \quad (18.50)$$

In the case of regression trees, we have  $F(\mathbf{x}) = w_{q(\mathbf{x})}$ , where  $q : \mathbb{R}^D \rightarrow \{1, \dots, J\}$  specifies which leaf node  $\mathbf{x}$  belongs to, and  $\mathbf{w} \in \mathbb{R}^J$  are the leaf weights. Hence we can rewrite Equation (18.49) as

2. Some other popular gradient boosted trees packages are **CatBoost** (<https://catboost.ai/>) and **LightGBM** (<https://github.com/Microsoft/LightGBM>).

follows, dropping terms that are independent of  $F_m$ :

$$\mathcal{L}_m(q, \mathbf{w}) \approx \sum_{i=1}^N \left[ g_{im} F_m(\mathbf{x}_i) + \frac{1}{2} h_{im} F_m^2(\mathbf{x}_i) \right] + \gamma J + \frac{1}{2} \lambda \sum_{j=1}^J w_j^2 \quad (18.51)$$

$$= \sum_{j=1}^J \left[ \left( \sum_{i \in I_j} g_{im} \right) w_j + \frac{1}{2} \left( \sum_{i \in I_j} h_{im} + \lambda \right) w_j^2 \right] + \gamma J \quad (18.52)$$

where  $I_j = \{i : q(\mathbf{x}_i) = j\}$  is the set of indices of data points assigned to the  $j$ 'th leaf.

Let us define  $G_{jm} = \sum_{i \in I_j} g_{im}$  and  $H_{jm} = \sum_{i \in I_j} h_{im}$ . Then the above simplifies to

$$\mathcal{L}_m(q, \mathbf{w}) = \sum_{j=1}^J \left[ G_{jm} w_j + \frac{1}{2} (H_{jm} + \lambda) w_j^2 \right] + \gamma J \quad (18.53)$$

This is a quadratic in each  $w_j$ , so the optimal weights are given by

$$w_j^* = -\frac{G_{jm}}{H_{jm} + \lambda} \quad (18.54)$$

The loss for evaluating different tree structures  $q$  then becomes

$$\mathcal{L}_m(q, \mathbf{w}^*) = -\frac{1}{2} \sum_{j=1}^J \frac{G_{jm}^2}{H_{jm} + \lambda} + \gamma J \quad (18.55)$$

We can greedily optimize this using a recursive node splitting procedure, as in Section 18.1. Specifically, for a given leaf  $j$ , we consider splitting it into a left and right half,  $I = I_L \cup I_R$ . We can compute the gain (reduction in loss) of such a split as follows:

$$\text{gain} = \frac{1}{2} \left[ \frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{(H_L + H_R) + \lambda} \right] - \gamma \quad (18.56)$$

where  $G_L = \sum_{i \in I_L} g_{im}$ ,  $G_R = \sum_{i \in I_R} g_{im}$ ,  $H_L = \sum_{i \in I_L} h_{im}$ , and  $H_R = \sum_{i \in I_R} h_{im}$ . Thus we see that it is not worth splitting a node if the gain is negative (i.e., the first term is less than  $\gamma$ ).

A fast approximation for evaluating this objective, that does not require sorting the features (for choosing the optimal threshold to split on), is described in [CG16].

## 18.6 Interpreting tree ensembles

Trees are popular because they are interpretable. Unfortunately, ensembles of trees (whether in the form of bagging, random forests, or boosting) lose that property. Fortunately, there are some simple methods we can use to interpret what function has been learned.

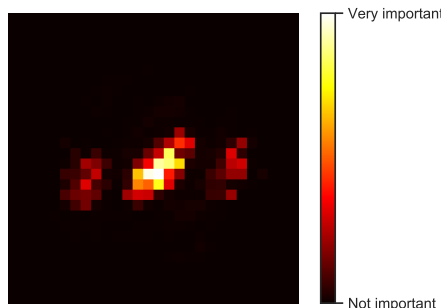


Figure 18.8: Feature importance of a random forest classifier trained to distinguish MNIST digits from classes 0 and 8. Adapted from Figure 7.6 of [Gér19]. Generated by `rf_feature_importance_mnist.ipynb`.

### 18.6.1 Feature importance

For a single decision tree  $T$ , [BFO84] proposed the following measure for **feature importance** of feature  $k$ :

$$R_k(T) = \sum_{j=1}^{J-1} G_j \mathbb{I}(v_j = k) \quad (18.57)$$

where the sum is over all non-leaf (internal) nodes,  $G_j$  is the gain in accuracy (reduction in cost) at node  $j$ , and  $v_j = k$  if node  $j$  uses feature  $k$ . We can get a more reliable estimate by averaging over all trees in the ensemble:

$$R_k = \frac{1}{M} \sum_{m=1}^M R_k(T_m) \quad (18.58)$$

After computing these scores, we can normalize them so the largest value is 100%. We give some examples below.

Figure 18.8 gives an example of estimating feature importance for a classifier trained to distinguish MNIST digits from classes 0 and 8. We see that it focuses on the parts of the image that differ between these classes.

In Figure 18.9, we plot the relative importance of each of the features for the spam dataset (Section 18.4). Not surprisingly, we find that the most important features are the words “george” (the name of the recipient) and “hp” (the company he worked for), as well as the characters ! and \$. (Note it can be the presence or absence of these features that is informative.)

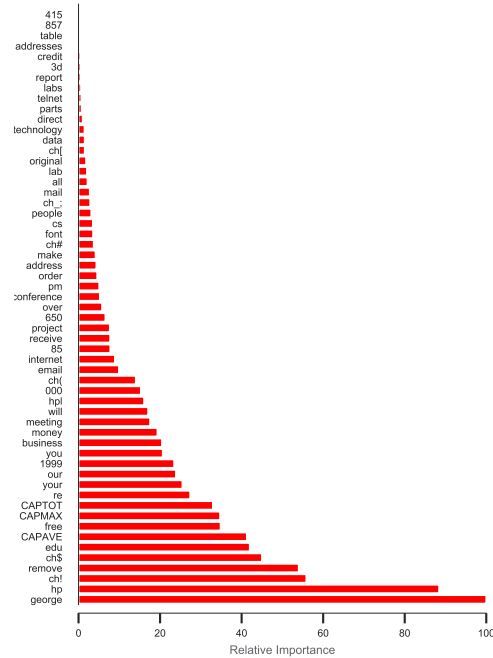


Figure 18.9: Feature importance of a gradient boosted classifier trained to distinguish spam from non-spam email. The dataset has  $X$  training examples with  $Y$  features, corresponding to token frequency. Adapted from Figure 10.6 of [HTF09]. Generated by `spam_tree_ensemble_interpret.ipynb`.

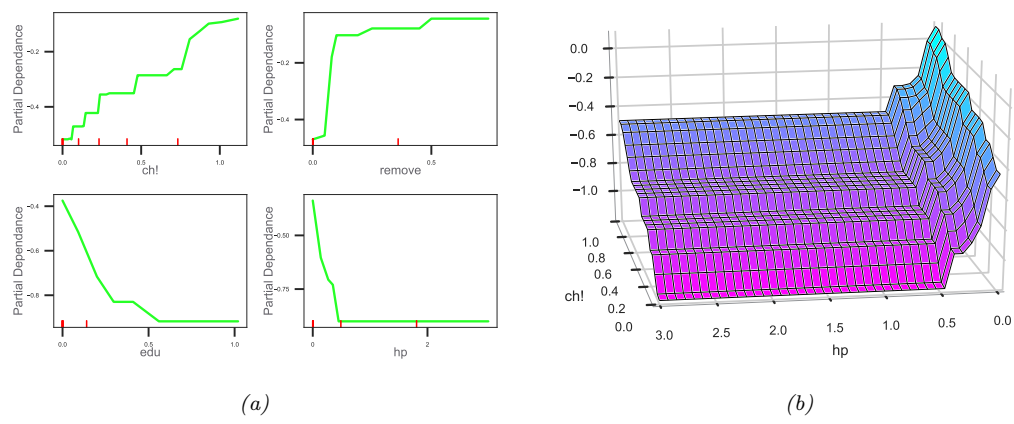


Figure 18.10: (a) Partial dependence of log-odds of the spam class for 4 important predictors. The red ticks at the base of the plot are deciles of the empirical distribution for this feature. (b) Joint partial dependence of log-odds on the features  $hp$  and  $!$ . Adapted from Figure 10.6–10.8 of [HTF09]. Generated by `spam_tree_ensemble_interpret.ipynb`.

### 18.6.2 Partial dependency plots

After we have identified the most relevant input features, we can try to assess the impact they have on the output. A **partial dependency plot** for feature  $k$  is a plot of

$$\bar{f}_k(x_k) = \frac{1}{N} \sum_{n=1}^N f(\mathbf{x}_{n,-k}, x_k) \quad (18.59)$$

vs  $x_k$ . Thus we marginalize out all features except  $k$ . In the case of a binary classifier, we can convert this to log odds,  $\log p(y=1|x_k)/p(y=0|x_k)$ , before plotting. We illustrate this for our spam example in Figure 18.10a for 4 different features. We see that as the frequency of “!” and “remove” increases, so does the probability of spam. Conversely, as the frequency of “edu” or “hp” increases, the probability of spam decreases.

We can also try to capture interaction effects between features  $j$  and  $k$  by computing

$$\bar{f}_{jk}(x_j, x_k) = \frac{1}{N} \sum_{n=1}^N f(\mathbf{x}_{n,-jk}, x_j, x_k) \quad (18.60)$$

We illustrate this for our spam example in Figure 18.10b for hp and !. We see that higher frequency of ! makes it more likely to be spam, but much more so if the word “hp” is missing.



PART V

# Beyond Supervised Learning



# 19 Learning with Fewer Labeled Examples

Many ML models, especially neural networks, often have many more parameters than we have labeled training examples. For example, a ResNet CNN (Section 14.3.4) with 50 layers has 23 million parameters. Transformer models (Section 15.5) can be even bigger. Of course these parameters are highly correlated, so they are not independent “degrees of freedom”. Nevertheless, such big models are slow to train and, more importantly, they may easily overfit. This is particularly a problem when you do not have a large labeled training set. In this chapter, we discuss some ways to tackle this issue, beyond the generic regularization techniques we discussed in Section 13.5 such as early stopping, weight decay and dropout.

## 19.1 Data augmentation

Suppose we just have a single small labeled dataset. In some cases, we may be able to create artificially modified versions of the input vectors, which capture the kinds of variations we expect to see at test time, while keeping the original labels unchanged. This is called **data augmentation**.<sup>1</sup> We give some examples below, and then discuss why this approach works.

### 19.1.1 Examples

For image classification tasks, standard data augmentation methods include random crops, zooms, and mirror image flips, as illustrated in Figure 19.1. [GVZ16] gives a more sophisticated example, where they render text characters onto an image in a realistic way, thereby creating a very large dataset of text “in the wild”. They used this to train a state of the art visual text localization and reading system. Other examples of data augmentation include artificially adding background noise to clean speech signals, and artificially replacing characters or words at random in text documents.

If we afford to train and test the model many times using different versions of the data, we can learn which augmentations work best, using blackbox optimization methods such as RL (see e.g., [Cub+19]) or Bayesian optimization (see e.g., [Lim+19]); this is called **AutoAugment**. We can also learn to combine multiple augmentations together; this is called **AutoAugment** [Cub+19].

For some examples of augmentation in NLP, see e.g., [Fen+21].

---

1. The term “data augmentation” is also used in statistics to mean the addition of auxiliary latent variables to a model in order to speed up convergence of posterior inference algorithms [DM01].



Figure 19.1: Illustration of random crops and zooms of the image on the left. Generated by `image_augmentation_jax.ipynb`.

### 19.1.2 Theoretical justification

Data augmentation often significantly improves performance (predictive accuracy, robustness, etc). At first this might seem like we are getting something for nothing, since we have not provided additional data. However, the data augmentation mechanism can be viewed as a way to algorithmically inject prior knowledge.

To see this, recall that in standard ERM training, we minimize the empirical risk

$$R(f) = \int \ell(f(\mathbf{x}), \mathbf{y}) p^*(\mathbf{x}, \mathbf{y}) d\mathbf{x} d\mathbf{y} \quad (19.1)$$

where we approximate  $p^*(\mathbf{x}, \mathbf{y})$  by the empirical distribution

$$p_{\mathcal{D}}(\mathbf{x}, \mathbf{y}) = \frac{1}{N} \sum_{n=1}^N \delta(\mathbf{x} - \mathbf{x}_n) \delta(\mathbf{y} - \mathbf{y}_n) \quad (19.2)$$

We can think of data augmentation as replacing the empirical distribution with the following algorithmically smoothed distribution

$$p_{\mathcal{D}}(\mathbf{x}, \mathbf{y} | A) = \frac{1}{N} \sum_{n=1}^N p(\mathbf{x} | \mathbf{x}_n, A) \delta(\mathbf{y} - \mathbf{y}_n) \quad (19.3)$$

where  $A$  is the data augmentation algorithm, which generates a sample  $\mathbf{x}$  from a training point  $\mathbf{x}_n$ , such that the label (“semantics”) is not changed. (A very simple example would be a Gaussian kernel,  $p(\mathbf{x} | \mathbf{x}_n, A) = \mathcal{N}(\mathbf{x} | \mathbf{x}_n, \sigma^2 \mathbf{I})$ .) This has been called **vicinal risk minimization** [Cha+01], since we are minimizing the risk in the vicinity of each training point  $\mathbf{x}$ . For more details on this perspective, see [Zha+17b; CDL19; Dao+19].

## 19.2 Transfer learning

*This section is coauthored with Colin Raffel.*

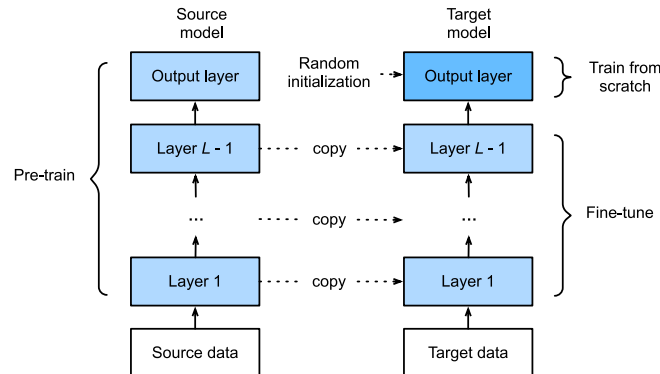


Figure 19.2: Illustration of fine-tuning a model on a new dataset. The final output layer is trained from scratch, since it might correspond to a different label set. The other layers are initialized at their previous parameters, and then optionally updated using a small learning rate. From Figure 13.2.1 of [Zha+20]. Used with kind permission of Aston Zhang.

Many data-poor tasks have some high-level structural similarity to other data-rich tasks. For example, consider the task of **fine-grained visual classification** of endangered bird species. Given that endangered birds are by definition rare, it is unlikely that a large quantity of diverse labeled images of these birds exist. However, birds bear many structural similarities across species - for example, most birds have wings, feathers, beaks, claws, etc. We therefore might expect that first training a model on a large dataset of non-endangered bird species and then continuing to train it on a small dataset of endangered species could produce better performance than training on the small dataset alone.

This is called **transfer learning**, since we are transferring information from one dataset to another, via a shared set of parameters. More precisely, we first perform a **pre-training phase**, in which we train a model with parameters  $\theta$  on a large **source dataset**  $\mathcal{D}_p$ ; this may be labeled or unlabeled. We then perform a second **fine-tuning phase** on the small labeled **target dataset**  $\mathcal{D}_q$  of interest. We discuss these two phases in more detail below, but for more information, see e.g., [Tan+18; Zhu+21] for recent surveys.

### 19.2.1 Fine-tuning

Suppose, for now, that we already have a pretrained classifier,  $p(y|\mathbf{x}, \theta_p)$ , such as a CNN, that works well for inputs  $\mathbf{x} \in \mathcal{X}_p$  (e.g. natural images) and outputs  $y \in \mathcal{Y}_p$  (e.g., ImageNet labels), where the data comes from a distribution  $p(\mathbf{x}, y)$  similar to the one used in training. Now we want to create a new model  $q(y|\mathbf{x}, \theta_q)$  that works well for inputs  $\mathbf{x} \in \mathcal{X}_q$  (e.g. bird images) and outputs  $y \in \mathcal{Y}_q$  (e.g., fine-grained bird labels), where the data comes from a distribution  $q(\mathbf{x}, y)$  which may be different from  $p$ .

We will assume that the set of possible inputs is the same, so  $\mathcal{X}_q \approx \mathcal{X}_p$  (e.g., both are RGB images), or that we can easily transform inputs from domain  $p$  to domain  $q$  (e.g., we can convert an RGB image to grayscale by dropping the chrominance channels and just keeping luminance). (If this is not

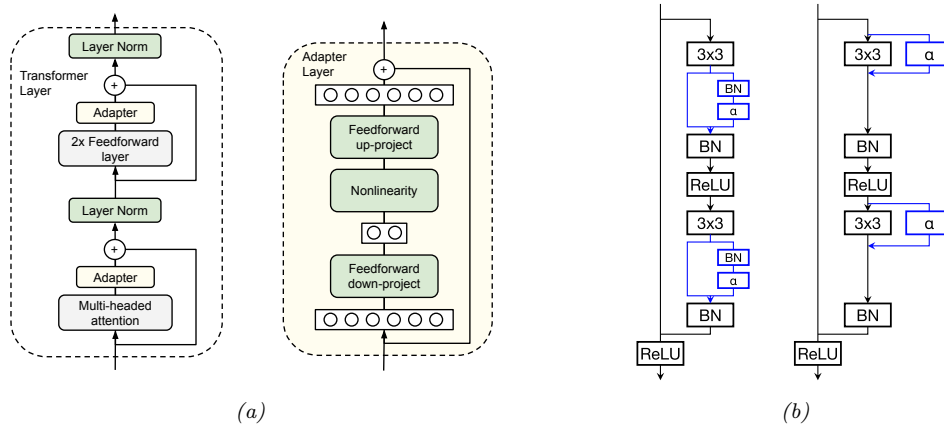


Figure 19.3: (a) Adding adapter layers to a transformer. From Figure 2 of [Hou+19]. Used with kind permission of Neil Houlsby. (b) Adding adapter layers to a resnet. From Figure 2 of [RBV18]. Used with kind permission of Sylvestre-Alvise Rebuffi.

the case, then we may need to use a method called domain adaptation, that modifies models to map between modalities, as discussed in Section 19.2.5.)

However, the output domains are usually different, i.e.,  $\mathcal{Y}_q \neq \mathcal{Y}_p$ . For example,  $\mathcal{Y}_p$  might be Imagenet labels and  $\mathcal{Y}_q$  might be medical labels (e.g., types of diabetic retinopathy [Arc+19]). In this case, we need to “translate” the output of the pre-trained model to the new domain. This is easy to do with neural networks: we simply “chop off” the final layer of the original model, and add a new “head” to model the new class labels, as illustrated in Figure 19.2. For example, suppose  $p(y|\mathbf{x}, \boldsymbol{\theta}_p) = \text{softmax}(y|\mathbf{W}_2\mathbf{h}(\mathbf{x}; \boldsymbol{\theta}_1) + \mathbf{b}_2)$ , where  $\boldsymbol{\theta}_p = (\mathbf{W}_2, \mathbf{b}_2, \boldsymbol{\theta}_1)$ . Then we can construct  $q(y|\boldsymbol{\theta}_q) = \text{softmax}(y|\mathbf{W}_3\mathbf{h}(\mathbf{x}; \boldsymbol{\theta}_1) + \mathbf{b}_3)$ , where  $\boldsymbol{\theta}_q = (\mathbf{W}_3, \mathbf{b}_3, \boldsymbol{\theta}_1)$  and  $\mathbf{h}(\mathbf{x}; \boldsymbol{\theta}_1)$  is the shared nonlinear feature extractor.

After performing this “model surgery”, we can fine-tune the new model with parameters  $\boldsymbol{\theta}_q = (\boldsymbol{\theta}_1, \boldsymbol{\theta}_3)$ , where  $\boldsymbol{\theta}_1$  parameterizes the feature extractor, and  $\boldsymbol{\theta}_3$  parameterizes the final linear layer that maps features to the new set of labels. If we treat  $\boldsymbol{\theta}_1$  as “frozen parameters”, then the resulting model  $q(y|\mathbf{x}, \boldsymbol{\theta}_q)$  is linear in its parameters, so we have a convex optimization problem for which many simple and efficient fitting methods exist (see Part II). This is particularly helpful in the long-tail setting, where some classes are very rare [Kan+20]. However, a linear “decoder” may be too limiting, so we can also allow  $\boldsymbol{\theta}_1$  to be fine-tuned as well, but using a lower learning rate, to prevent the values moving too far from the values estimated on  $\mathcal{D}_p$ .

## 19.2.2 Adapters

One disadvantage of fine-tuning all the model parameters of a pre-trained model is that it can be slow, since there are often many parameters, and we may need to use a small learning rate to prevent the low-level feature extractors from diverging too far from their prior values. In addition, every new task requires a new model to be trained, making task sharing hard. An alternative approach is to keep the pre-trained model untouched, but to add new parameters to modify its internal behavior to